Python 函数核心用法学习笔记

本文档记录了 Python 函数从基础定义、用户交互，到复杂参数混用（`*args`, `**kwargs`）以及多值返回的核心机制与代码示例。

---

1. 基础函数与用户输入

函数能够接收外部传入的数据，并结合 `f-string` 实现灵活的格式化输出。我们也可以将用户的实时输入（`input`）直接传递给函数。

```
# 示例 1：最基础的函数调用
def my_function_basic(name, lvl):
    """这是一个简单的函数示例"""
    print("Hello, World!", name, "Your level is", lvl)

my_function_basic("Alice", 5)

# 示例 2：结合 input() 获取用户输入并传参
def my_function_role(name, gender, lvl, role):
    print(f"玩家：{name}，性别：{gender}，等级：{lvl}，职业：{role}")

# 实际运行时可取消注释以下代码：
# name = input("请输入玩家姓名：")
# gender = input("请输入玩家性别：")
# lvl = int(input("请输入玩家等级："))
# role = input("请输入玩家职业：")
# my_function_role(name, gender, lvl, role)
```

---

2. 动态参数与黑洞机制

当不确定会传入多少个参数时，Python 提供了强大的动态参数机制。这也是本节学习的核心难点：**参数的排队铁律**。

四种参数混用示例

在调用时，无名氏数据（位置实参）必须永远排在带等号的数据（关键字实参）前面。

```
# 1. 仅使用 **kwargs（接收带等号的参数，打包为字典）
def build_profile_simple(name, **info):
    print(f"姓名：{name}")
    print(f"其他信息：{info}")

build_profile_simple("Alice", age=25, city="上海", job="法师")
# 输出：其他信息：{'age': 25, 'city': '上海', 'job': '法师'}
```

```python
# 2. 终极完全体：位置参数 + *args + 默认参数 + **kwargs
def build_profile_full(name, *sss, lvl=0, **info):
    print(f"姓名：{name}等级：{lvl}", sss, info)

# 调用时：1, 2, 3 被 *sss 吸收；role, age, city, job 被 **info 吸收
build_profile_full("Alice", 5, 1, 2, 3, role="战士", age=25, lvl=10, city="上海",
job="法师")

# 3. 业界标准写法：*args 与 **kwargs
def bai_fen(*args, **kwargs):
    print("位置参数打包(元组):", args)
    print("关键字参数打包(字典):", kwargs)

bai_fen(1, 2, 3, name="Alice", age=25)
```

参数出场顺序速查表

出场顺序	参数类型	代码长相	底层机制
第 1 位	位置参数	`name, lvl`	严格按顺序、一对一接收传入的数据。
第 2 位	动态位置参数	`*args` / `*sss`	像黑洞一样吸收剩余的无名数据，打包成**元组** `()`。
第 3 位	默认参数	`lvl=0`	带有默认值，调用时可传可不传。
第 4 位	动态关键字参数	`**kwargs` / `**info`	吸收所有带等号的 VIP 数据，打包成**字典** `{}`。

> ⚠ **避坑指南**：在编写复杂函数时，严格遵守上述顺序可以避免绝大多数 `SyntaxError` 报错。

---

3. 列表解包 (Unpacking)

如果在函数外部已经有了一个列表，想要把它里面的元素当作独立参数传给函数，可以在列表前加上 `*` 直接“打散”。

```python
ori_list = [1, 2, 3, 4, 5]

def modify_list(*a):
    print("接收到的原始列表被打散后重新打包:", a)

# 使用 * 号将列表解包传入
modify_list(*ori_list)
```

---

4. 返回值：单值与多值打包

函数不仅能处理数据，还能将结果 `return` 给外部变量。Python 拥有独特的**自动打包**特权，允许同时返回多个值。

```python
# 示例 1：常规的单值返回
def func_single(a, d):
    return a + d

result = func_single(5, 10)
print("单值返回结果:", result)  # 输出: 15

# 示例 2：多值返回与自动打包 (Tuple Packing)
def func_multi(a, d):
    return a, 1, 2  # 用逗号隔开，同时返回三个独立数据

function = func_multi(5, 10)
print("多值返回结果:", function)  # 输出: (5, 1, 2)
```

原理解析： 当 `return` 后面跟着多个用逗号隔开的值时，Python 会自动找一个“快递盒”（元组 Tuple）把它们装起来，作为一个整体交出。